

Dev Learning Path

Guia de estudos para quem esta começando no desenvolvimento

Este guia cobre os topicos fundamentais que todo desenvolvedor precisa dominar, organizados em uma progressao logica — do mais basico ao mais avancado. Cada fase depende da anterior. Siga a ordem.

Phase 1

Fundamentos

Antes de qualquer pattern ou segurança, entender como a web funciona de verdade. Sem isso, nada do que vem depois vai fazer sentido.

Frontend vs Backend

O que cada lado faz e como se comunicam

- Cliente (navegador) envia requests, servidor processa e responde
- Frontend: HTML, CSS, JS — o que o usuario ve
- Backend: logica de negocio, banco de dados, APIs
- Comunicacao via HTTP — sempre texto, sempre stateless

HTTP & REST

Request, response, status codes, verbos

- GET, POST, PUT, PATCH, DELETE — cada um tem proposito
- Status codes: 2xx sucesso, 4xx erro do cliente, 5xx erro do servidor
- Headers carregam metadados: Content-Type, Authorization, etc.
- REST: recursos nomeados por URL, acoes pelos verbos HTTP

CRUD

Create, Read, Update, Delete — na pratica

- Toda aplicacao basicamente cria, le, edita e deleta dados
- Create -> POST, Read -> GET, Update -> PUT/PATCH, Delete -> DELETE
- Mapeia diretamente pro banco: INSERT, SELECT, UPDATE, DELETE
- Entender CRUD e a base de qualquer feature que voce vai construir

JSON & APIs

Estrutura de dados, fetch, integracao

- JSON e o formato padrao de troca de dados na web
- Chave-valor, arrays, objetos aninhados — sempre texto
- fetch() no frontend, axios, ou qualquer cliente HTTP no backend
- Leia a documentacao da API antes de codar — sempre

Phase 2

Clean Code & Patterns

Codigo que funciona nao e o suficiente. Codigo que outra pessoa consegue ler, manter e estender — isso e o objetivo.

Clean Code <i>Nomes, funcoes, responsabilidade unica</i> • Nomes descritivos: <code>getUserById()</code>, <code>getData()</code> • Funcoes pequenas, que fazem uma coisa so • Evite comentarios obvios — o codigo deve se explicar • DRY: Don't Repeat Yourself — abstraia duplicacao	SOLID <i>5 principios de design de codigo</i> • S: Single Responsibility — uma classe, uma razao pra mudar • O: Open/Closed — aberto pra extensao, fechado pra modificacao • L: Liskov — subclasses devem ser substituiveis pela classe pai • I/D: Interfaces coesas, dependa de abstracoes nao de concretos
Design Patterns <i>Repository, Service, Factory, Strategy</i> • Repository: isola o acesso ao banco do resto do codigo • Service: logica de negocio fica aqui, nao no controller • Factory: cria objetos sem expor a logica de criacao • Strategy: troca algoritmos em runtime sem mudar o cliente	MVC & Camadas <i>Controller, Service, Repository</i> • Controller: recebe request, valida entrada, chama Service • Service: logica de negocio, orquestra Repository e outros servicos • Repository: unico ponto de acesso ao banco de dados • Separar camadas facilita testes e manutencao

Phase 3

Seguranca — Auth

Seguranca nao e feature, e requisito. Entender autenticacao e autorizacao e obrigatorio antes de colocar qualquer app em producao.

Authn vs Authz <i>Quem e voce? O que voce pode fazer?</i> • Autenticacao: verifica identidade (login, senha, token) • Autorizacao: verifica permissao (pode acessar este recurso?) • Sao coisas distintas — confundir gera brechas de seguranca • Exemplo: voce esta logado (authn) mas nao e admin (authz)	JWT & Tokens <i>Header, payload, signature, refresh</i> • JWT = Header.Payload.Signature — tudo em Base64 • Stateless: o servidor nao precisa guardar sessao • Access token: vida curta (15min). Refresh token: vida longa • Nunca guarde JWT em localStorage — use httpOnly cookie
OAuth2 & OIDC <i>Login social, fluxos de autorizacao</i> • OAuth2: framework de autorizacao (nao autenticacao) • OIDC adiciona camada de identidade sobre OAuth2 • Fluxos: Authorization Code (web), Client Credentials (M2M) • Use uma biblioteca consolidada — nao implemente do zero	OWASP Top 10 <i>XSS, SQL Injection, CSRF e outros</i> • SQL Injection: nunca concatene input direto em queries • XSS: nunca renderize HTML de input sem sanitizar • CSRF: use tokens CSRF em forms que modificam estado • Leia o OWASP Top 10 — e a lista das vulnerabilidades mais comuns

Phase 4

Testes

Testes não são extras — são a rede de segurança que permite você mudar código com confiança. Sem testes, cada mudança é um risco.

Pirâmide de Testes

Unit, integration, e2e

- Unit: testa uma função isolada, rápido, barato, muitos
- Integration: testa módulos integrados (ex: service + banco)
- E2E: testa fluxo completo, lento, caro, poucos
- Mais unit tests na base, menos e2e no topo — proposital

Unit Tests

Arrange, Act, Assert — mocks, stubs

- Arrange: prepara o estado inicial e dependências
- Act: executa a função sendo testada
- Assert: verifica o resultado esperado
- Mocks substituem dependências externas (banco, API, etc.)

TDD

Red, Green, Refactor

- Red: escreve o teste primeiro — ele vai falhar
- Green: escreve o código mínimo para o teste passar
- Refactor: melhora o código sem quebrar o teste
- TDD força você a pensar na interface antes da implementação

API Testing

Integration tests, contratos

- Testa endpoint real: request -> response -> status -> body
- Verifica contratos: a API retorna o que prometeu?
- Use ferramentas como Supertest (Node) ou TestClient (FastAPI)
- Testa casos de erro também — não só o happy path

Phase 5

Spec Driven Development

Com Cursor e AI, quem escreve specs claras ganha muito mais do que quem só descreve o problema vagamente. A qualidade do output é proporcional à qualidade da spec.

O que é SDD

Escrever specs antes de codificar

- Spec = descrição técnica detalhada do que vai ser construído
- Escrever a spec força você a pensar antes de codar
- Reduz retrabalho: descobrir problemas na spec é mais barato
- Com AI: spec boa = código gerado muito melhor

Como escrever specs

Formato, nível de detalhe, exemplos

- Contexto: qual problema resolve, quem usa, quais restrições
- Endpoints: método, path, request body, response, erros possíveis
- Regras de negócio: o que deve e o que não deve acontecer
- Critérios de aceite: como saber que está pronto e correto

OpenAPI / Swagger <i>Contrato da API antes do código</i> • Define endpoints, parâmetros, schemas e respostas em YAML • Gera documentação automática e cliente de testes (Swagger UI) • Pode ser usado pra gerar código de cliente e servidor • Escreva o OpenAPI spec antes de implementar — e a spec da API	Cursor + Specs <i>Prompts melhores com contexto claro</i> • Ruim: 'cria um endpoint de login' • Bom: 'cria POST /auth/login que recebe email e password, valida com bcrypt, retorna JWT de 15min e refresh token de 7 dias' • Inclua restrições, erros esperados e exemplos no prompt • Quanto mais contexto e precisão, menos retrabalho
--	---

Notas importantes

Por que essa ordem? Fundamentos primeiro porque sem entender HTTP e CRUD, patterns e JWT não fazem sentido. Clean code antes de segurança porque código organizado é mais fácil de auditar. Testes antes de SDD porque você precisa saber o que testar pra escrever specs com critérios de aceite.

Sobre usar Cursor no início O risco de aprender dev com AI é que o código gerado parece certo mas você não sabe por que. Use o Cursor pra acelerar, mas entenda o que foi gerado. Se não entender, pergunte ao Cursor — use ele como professor, não só como executor.

Projetos pra praticar Construa uma API de tarefas completa (CRUD + autenticação JWT + testes). É simples o suficiente pra caber numa semana, mas cobre todos os tópicos do path.